

SQL Questions 1–6: Questions and Solutions

Each question is followed by its SQL solution.

SQL 1 - Easy: Customer with the Highest Value Order

Question: Management wants to know which customer placed the single highest-value order.

Write a SQL query using a single-row subquery to return the order_id, customer_name, and total_amount for the order with the maximum total_amount in the Orders table.

SQL Solution:

```
USE retail_store;
```

```
SELECT o.order_id, c.name AS customer_name, MAX(total_amount) AS total_amount
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY o.order_id
ORDER BY total_amount DESC
LIMIT 1;
```

OR

```
SELECT o.order_id, c.name AS customer_name, o.total_amount
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
WHERE o.total_amount = (
    SELECT MAX(total_amount) FROM Orders
);
```

SQL 2 - Easy: Student Enrollments with Grades and Department Names

Question: Generate a report listing every student along with the courses they are enrolled in, the grades they have received, and the departments offering those courses.

Return these columns:

student_id, student_name, course_id, course_name, grade, department_name.

SQL Solution:

```
USE university_core;

SELECT s.student_id,
       s.name AS student_name,
       c.course_id,
       c.course_name,
       e.grade,
       d.department_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
JOIN Departments d ON c.department_id = d.department_id
ORDER BY s.student_id, c.course_id;
```

SQL 3 - Easy: Calculate the Average Assignment Score per Course

Question: Calculate the average assignment score for each course.
Join the suitable tables to obtain the course name and average score.
Display the results in descending order of average score.

SQL Solution:

```
USE online_learning;

SELECT c.course_name, AVG(s.score) AS avg_score
FROM Courses c
JOIN Assignments a ON c.course_id = a.course_id
JOIN Submissions s ON a.assignment_id = s.assignment_id
GROUP BY c.course_id, c.course_name
ORDER BY avg_score DESC;
```

SQL 4 - Medium: Highest-Revenue Doctor per Department

Question: A hospital administration system tracks appointments and billing for doctors across departments. Generate a report showing each doctor's department name, doctor name, total revenue from all bills, and the doctor's rank within the department.

Use RANK(), not ROW_NUMBER(), so doctors with equal revenue receive the same rank.

Return: department_name, doctor_name, total_revenue, revenue_rank.

SQL Solution:

```
USE HOSPITAL_DB;

SELECT d.department_name,
       doc.doctor_name,
       SUM(b.amount) AS total_revenue,
       RANK() OVER (
           PARTITION BY d.dept_id
           ORDER BY SUM(b.amount) DESC
       ) AS revenue_rank
FROM Departments d
JOIN Doctors doc ON d.dept_id = doc.dept_id
JOIN Appointments a ON doc.doctor_id = a.doctor_id
JOIN Bills b ON a.appointment_id = b.appointment_id
GROUP BY d.department_name, d.dept_id, doc.doctor_name
ORDER BY d.dept_id, revenue_rank;
```

SQL 5 - Medium: Students Ranked by Course Enrollments

Question: Write a query to assign a RANK and a DENSE_RANK to each student based on the number of courses they are enrolled in, in descending order. Students with the same number of enrollments must receive the same rank.

Return: student_id, name, course_count, rank_by_enrollments, dense_rank_by_enrollments.
Order the final result by rank_by_enrollments.

SQL Solution:

```
USE university_core;

SELECT s.student_id,
       s.name,
       COUNT(e.course_id) AS course_count,
       RANK() OVER (
         ORDER BY COUNT(e.course_id) DESC
       ) AS rank_by_enrollments,
       DENSE_RANK() OVER (
         ORDER BY COUNT(e.course_id) DESC
       ) AS dense_rank_by_enrollments
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY rank_by_enrollments;
```

SQL 6 - Medium: Movies Watched by Viewers of 'Life of Pi'

Question: List all movies that have been watched by any user who has watched 'Life of Pi'. Duplicate movie titles must not appear in the result.

Return only movie_id and title, and include movies watched by users who also watched 'Life of Pi'.

SQL Solution:

```
USE streaming_platform;

SELECT DISTINCT m.movie_id, m.title
FROM Movies m
JOIN Watch_History w ON m.movie_id = w.movie_id
WHERE w.user_id IN (
    SELECT user_id
    FROM Watch_History
    WHERE movie_id = 902
);
```